

MARK HANEY

STAFF SOFTWARE ENGINEER | PYTHON AI/ML ENGINEER | ML PLATFORM LEAD | CLOUD, MLOPS, FASTAPI, PYTORCH

Dardanelle, AR

<https://www.linkedin.com/in/markhaney-33b675ba>

+1 (430) 279-2231 | markhaney.dev@outlook.com

PROFESSIONAL SUMMARY

Staff Software Engineer specializing in Python AI/ML platforms for SaaS, PropTech, nonprofit CRM, GovTech, and managed IT industries, using **Python 3.8–3.12**, **FastAPI 0.95–0.111**, **PyTorch 1.13–2.3**, **TensorFlow 2.8–2.16**, **scikit-learn 1.0–1.5**, **Pandas 1.3–2.2**, **Airflow 2.3–2.9**, **Docker 20–26**, **Kubernetes 1.23–1.30**, and AWS/GCP cloud services. Strong background turning data-heavy products into secure ML services, building feature pipelines, deploying model APIs, improving inference reliability, reducing batch latency, migrating legacy Python services, and mentoring teams through clean architecture, testing, CI/CD, observability, monitoring, and production support.

TECH SKILLS

- ❖ **Programming:** Python 3.8–3.12, SQL, Bash, TypeScript, JavaScript
- ❖ **AI/ML:** PyTorch 1.13–2.3, TensorFlow 2.8–2.16, scikit-learn 1.0–1.5, XGBoost 1.6–2.0, LightGBM 3.3–4.3, Hugging Face Transformers 4.25–4.41
- ❖ **LLM / GenAI:** RAG, embeddings, prompt evaluation, vector search, LangChain 0.0.300–0.2, LlamaIndex 0.8–0.10
- ❖ **Backend:** FastAPI 0.95–0.111, Django 3.2–5.0, Flask 2.0–3.0, REST APIs, async services, background jobs
- ❖ **Data Engineering:** Pandas 1.3–2.2, NumPy 1.21–1.26, PySpark 3.2–3.5, Airflow 2.3–2.9, dbt 1.4–1.8
- ❖ **MLOps:** MLflow 2.3–2.13, model registry, feature pipelines, inference monitoring, A/B testing, drift checks
- ❖ **Cloud / DevOps:** AWS, GCP, Docker 20–26, Kubernetes 1.23–1.30, Terraform 1.3–1.8, GitHub Actions, CI/CD
- ❖ **Databases:** PostgreSQL 12–16, MongoDB 4.4–7.0, Redis 6–7, BigQuery, Snowflake, Elasticsearch 7–8
- ❖ **Testing:** Pytest 6–8, unit testing, integration testing, API contract testing, load testing, model validation

PROFESSIONAL EXPERIENCE

Staff Software Engineer

Relatio

Jun 2022 – Apr 2026

Las Vegas, NV

- ❖ Led the design of a **Python 3.10–3.12 AI/ML platform** for a SaaS data workflow product, connecting FastAPI services, ML inference APIs, feature pipelines, and cloud-native deployment patterns.
- ❖ Built production-ready **FastAPI 0.95–0.111** services around PyTorch and scikit-learn models, improving model response consistency and reducing average inference latency by **32%**.
- ❖ Designed reusable feature extraction pipelines with **Pandas 1.5–2.2**, NumPy, and Airflow, reducing duplicate preprocessing logic across teams by **26%**.
- ❖ Implemented a **RAG-based knowledge search feature** using embeddings, vector search, and prompt evaluation workflows to help internal users retrieve context from complex product data.

- ❖ Resolved a recurring production issue where model APIs returned inconsistent predictions because training and inference used different normalization logic, then centralized the feature contract.
- ❖ Migrated legacy synchronous Python services into async **FastAPI** endpoints with background workers, improving request throughput by **29%** under high-volume data processing workloads.
- ❖ Improved model observability by adding structured logs, latency metrics, drift checks, and failure dashboards across Kubernetes workloads and CI/CD release pipelines.
- ❖ Created a model validation layer that blocked low-quality deployments when accuracy, precision, recall, or input schema checks failed during automated release testing.
- ❖ Reworked Docker images and dependency management for **Python 3.11**, reducing container size and improving cold-start behavior by **21%** in staging and production.
- ❖ Introduced reusable **MLflow 2.x** experiment tracking and model registry patterns so engineers could compare model versions without relying on manual spreadsheets.
- ❖ Mentored backend and data engineers on Python testing, model API design, CI/CD quality gates, and safe rollout strategies for AI-powered product features.
- ❖ Partnered with product and data stakeholders to convert unclear business requirements into measurable ML tasks, clear acceptance criteria, and reliable production delivery.

Senior Software Engineer

AppFolio

Feb 2018 – May 2022

Goleta, CA

- ❖ Built Python-based machine learning services for a **PropTech SaaS platform**, using Python 3.7–3.10, Django, Flask, scikit-learn, TensorFlow 2.x, PostgreSQL, and AWS.
 - ❖ Developed predictive scoring pipelines for property and customer workflow data, reducing manual review effort by **35%** through cleaner feature engineering and automated ranking.
 - ❖ Improved classification precision by **18%** after identifying noisy labels, adding feature validation, and retraining models with balanced sampling strategies.
 - ❖ Migrated older Python 2.7 utility scripts into **Python 3.8–3.10**, replacing fragile cron jobs with containerized batch services and tested workflows.
 - ❖ Solved a recurring batch failure where malformed CSV uploads broke downstream model jobs, then added schema checks, quarantine storage, and retry-safe processing.
 - ❖ Built REST APIs that exposed ML predictions to internal product features while protecting sensitive tenant data through authorization, audit logging, and input validation.
 - ❖ Reduced AWS processing cost by **22%** by optimizing batch window sizes, caching repeated lookups, and removing unnecessary model recomputation.
 - ❖ Implemented CI/CD checks with Pytest, Docker, and GitHub Actions so backend services and data pipelines were validated before production release.
 - ❖ Collaborated with product managers and support teams to investigate prediction quality issues, explain model behavior, and prioritize fixes based on customer impact.
 - ❖ Introduced shared coding standards for Python services, improving maintainability across backend, data engineering, and analytics-focused development teams.
-

Software Engineer

Blackbaud

Jul 2016 – Jan 2018

Charleston, SC

- ❖ Developed Python and data-processing features for a **nonprofit CRM and fundraising platform**, using Python 3.5–3.6, Django, Celery, PostgreSQL, Redis, and scikit-learn.
- ❖ Built donor matching and record deduplication workflows with **scikit-learn 0.18–0.19**, reducing duplicate constituent records by **27%** across imported customer datasets.
- ❖ Improved nightly data processing reliability by **31%** after tracing Celery queue failures to timeout handling, memory-heavy joins, and missing retry logic.
- ❖ Implemented campaign segmentation logic that used historical donation behavior, engagement signals, and rule-based scoring to improve targeting accuracy by **19%**.
- ❖ Migrated legacy data import modules into cleaner service classes, adding unit tests and integration checks around validation, transformation, and persistence layers.
- ❖ Fixed a production issue where duplicate background jobs created inconsistent donor records by adding idempotency keys, Redis locks, and safer transaction boundaries.
- ❖ Created internal dashboards for processing errors, rejected records, and job duration trends so support teams could diagnose customer import issues faster.
- ❖ Worked across backend, QA, and customer success teams to translate fundraising workflow problems into reliable Python services and measurable product improvements.
- ❖ Strengthened test coverage around financial and donor-data workflows, reducing regression risk in high-impact CRM features used by nonprofit customers.

Software Developer

Granicus

May 2014 – Jun 2016

Dallas, TX

- ❖ Built backend and data automation tools for a **GovTech communication platform**, using Python 2.7–3.5, Django 1.7–1.9, PostgreSQL, SQL Server, and Jenkins.
 - ❖ Developed document classification and citizen communication workflows with **scikit-learn 0.16–0.17**, improving routing accuracy by **21%** for high-volume public-sector content.
 - ❖ Reduced manual reporting time by **34%** by building Python ETL scripts that cleaned, aggregated, and exported operational metrics from multiple government systems.
 - ❖ Migrated older procedural scripts into maintainable Django service modules, adding validation, logging, and safer database access patterns for production jobs.
 - ❖ Fixed recurring encoding errors in imported government documents by standardizing UTF-8 handling, adding fallback parsing, and isolating failed records for review.
 - ❖ Improved deployment stability by **17%** after adding Jenkins validation steps, smoke tests, and rollback documentation for backend service releases.
 - ❖ Collaborated with product owners to implement new reporting features that helped public agencies track message delivery, user engagement, and compliance activity.
 - ❖ Supported production incidents by reading logs, reproducing issues locally, patching Python services, and documenting root-cause fixes for future maintenance.
-

Junior Software Developer

Vology

Jun 2013 – Apr 2014

Gainesville, FL

- ❖ Built internal automation tools for a **managed IT and cloud services company**, using Python 2.7, Django 1.5–1.6, MySQL, Bash, and early scikit-learn workflows.
- ❖ Created ticket triage scripts that grouped support requests by keywords, service category, and customer priority, reducing manual routing time by **23%**.
- ❖ Improved inventory forecasting reports by **16%** using Python data scripts, SQL queries, and basic statistical models for hardware and service demand tracking.
- ❖ Fixed recurring automation failures by adding input validation, better exception handling, and logging around file parsing, database updates, and scheduled jobs.
- ❖ Supported senior engineers with backend bug fixes, unit testing, deployment scripts, and documentation while building stronger Python, SQL, and troubleshooting skills.

EDUCATION

Arkansas State University

Bachelor's degree, Computer Science (Sep 2009 – May 2013)